BÁO CÁO THỰC HÀNH LAB 19

GIT RECOVERY

Người báo cáo: Liamz07

1. Mục tiêu:

- **Hoàn tác thay đổi cục bộ (git restore):** Hủy bỏ các chỉnh sửa chưa commit tại Working Directory hoặc gỡ tệp khỏi Staging Area.
- **Lưu trữ tạm thời (git stash):** Cất giữ nhanh các thay đổi đang làm dở để chuyển sang nhánh khác xử lý công việc khẩn cấp mà không cần tạo commit rác.
- **Đảo ngược & Đặt lại lịch sử (git revert vs git reset):** Phân biệt rõ cơ chế hoàn tác an toàn cho nhánh chung (revert) và cơ chế quay ngược con trỏ lịch sử (reset với các cờ --soft, --mixed, --hard).
- **Khôi phục dữ liệu bị mất (git reflog):** Sử dụng nhật ký tham chiếu nội bộ của Git để tìm lại và phục hồi các commit đã bị xóa nhầm do git reset --hard.

2. Môi trường và chuẩn bị:

- **Hệ điều hành:** Ubuntu Linux / macOS / Windows (sử dụng Terminal / Git Bash)
- **Công cụ:** Git CLI (Phiên bản 2.23+ trở lên để hỗ trợ đầy đủ lệnh git restore).
- **Khởi tạo môi trường thực hành:**

```
mkdir git-recovery-lab
cd git-recovery-lab
git init

# Tạo 2 commit mốc ban đầu
echo "Version 1.0" > app.txt
git add app.txt
git commit -m "Commit 1: Initial version 1.0"

echo "Version 2.0" >> app.txt
git add app.txt
git commit -m "Commit 2: Update to version 2.0"
```

3. Nội dung thực hành chi tiết:

BÀI 1: HOÀN TÁC THAY ĐỔI CHƯA COMMIT VỚI GIT RESTORE

- **Mục đích:** Khôi phục tệp tin về trạng thái sạch sẽ của lần commit gần nhất.
- **Thao tác thực thi:**

```
mkdir git-conflict-lab
cd git-conflict-lab
git init

# Tạo tệp tin ban đầu
echo "Original content on main branch" > config.txt
git add config.txt
git commit -m "Initial commit with config.txt"
```

- **Kết quả:** Tệp app.txt trở về nguyên vẹn nội dung Version 2.0 mà không làm bẩn lịch sử commit.

BÀI 2: CẤT GIỮ CÔNG VIỆC ĐANG LÀM DỞ VỚI GIT STASH

- **Mục đích:** Lưu tạm thay đổi chưa hoàn thiện vào bộ nhớ đệm (Stash stack) để chuyển nhánh gấp.
- **Thao tác thực thi:**

```
# Đang viết dở tính năng mới
echo "Work in progress feature..." >> app.txt

# Cất tạm thay đổi vào Stash kèm ghi chú
git stash push -m "WIP: half-baked feature"

# Kiểm tra trạng thái (Working directory hoàn toàn sạch)
git status

# Xem danh sách các bản lưu tạm
git stash list

# Lấy lại công việc đang làm dở ra tiếp tục và xóa khỏi Stash
git stash pop
```

- **Kết quả:** Toàn bộ dòng "Work in progress feature..." được khôi phục nguyên trạng về Working Directory sau khi chạy git stash pop.

BÀI 3: ĐẢO NGƯỢC COMMIT AN TOÀN VỚI GIT REVERT

- **Mục đích:** Hủy bỏ tác động của một commit đã tồn tại bằng cách tạo ra một commit mới mang nội dung ngược lại (giữ nguyên lịch sử cũ, cực kỳ an toàn khi làm việc nhóm).
- **Thao tác thực thi:**

```
# Dọn sạch working directory trước khi test
git restore app.txt

# Tạo một commit chứa lỗi (Commit 3)
echo "Buggy feature in v3.0" >> app.txt
git add app.txt
git commit -m "Commit 3: Add buggy feature"

# Dùng git revert để đảo ngược Commit 3 (HEAD)
git revert HEAD --no-edit

# Kiểm tra lịch sử và nội dung tệp
git log --oneline
cat app.txt
```

- **Kết quả:** Lịch sử xuất hiện thêm Commit 4: Revert "Commit 3: Add buggy feature", nội dung tệp app.txt tự động loại bỏ dòng "Buggy feature in v3.0".

BÀI 4: QUAY NGƯỢC CON TRỎ LỊCH SỬ VỚI GIT RESET

- **Mục đích:** Di chuyển con trỏ HEAD lùi về một commit cũ hơn trong quá khứ.
- **Phân biệt 3 chế độ hoạt động:**
  - *git reset --soft HEAD~1:* Lùi lại 1 commit, giữ nguyên thay đổi ở Staging Area.
  - *git reset --mixed HEAD~1* (mặc định)*:* Lùi lại 1 commit, đưa thay đổi về Working Directory.
  - *git reset --hard HEAD~1:* Lùi lại 1 commit và xóa sạch toàn bộ thay đổi ở cả Staging lẫn Working Directory.

4. Kịch bản làm mất commit và khôi phục bằng git reflog:

Đây là tình huống cứu hộ dữ liệu quan trọng nhất trong quản trị mã nguồn khi lập trình viên vô tình dùng lệnh xóa cứng (--hard) làm biến mất hoàn toàn lịch sử commit trên git log.

BƯỚC 1: TẠO DỮ LIỆU QUAN TRỌNG VÀ CỐ TÌNH LÀM MẤT BẰNG GIT RESET --HARD

```
# Tạo một commit chứa tính năng cực kỳ quan trọng
echo "CRITICAL PRODUCTION FEATURE - DO NOT LOSE" > important.txt
git add important.txt
git commit -m "Commit 5: Add critical production feature"

# Kiểm tra lịch sử hiện tại (Ghi nhận mã hash của Commit 5)
git log --oneline

# Sự Cố: Vô tình chạy reset --hard lùi lại 2 commit
git reset --hard HEAD~2

# Kiểm tra lại git log và thư mục
git log --oneline
ls -la
```

- **Hiện tượng quan sát:**
  - *Commit 5: Add critical production feature* hoàn toàn biến mất khỏi *git log*.
  - Tệp *important.txt* bị xóa sổ khỏi thư mục làm việc.

BƯỚC 2: TRUY VẾT DẤU VẾT TRONG NHẬT KÝ THAM CHIẾU (GIT REFLOG)

- **Nguyên lý kỹ thuật:** Mặc dù git log không còn hiển thị Commit 5, nhưng Git vẫn lưu giữ mọi chuyển động của con trỏ HEAD trong vùng nhật ký cục bộ mang tên Reference Logs (reflog) trong vòng mặc định 90 ngày trước khi trình thu gom rác (Garbage Collector) xóa vĩnh viễn.
- **Thao tác thực thi:** git reflog
- **Kết quả hiển thị trên Terminal:**

```
8f3a1b2 (HEAD -> main) HEAD@{0}: reset: moving to HEAD~2
c9d4e5f HEAD@{1}: commit: Commit 5: Add critical production feature
a1b2c3d HEAD@{2}: revert: Revert "Commit 3: Add buggy feature"
...
```

- **Phân tích:** Dòng HEAD@{1} chính là thời điểm ngay trước khi lệnh reset --hard xảy ra, mang mã định danh SHA-1 của commit bị mất là c9d4e5f.

BƯỚC 3: PHỤC HỒI NGUYÊN VẸN COMMIT VÀ DỮ LIỆU BỊ MẤT

- **Sử dụng mã băm c9d4e5f (hoặc định danh HEAD@{1}) để đưa nhánh main quay trở lại đúng thời điểm trước khi xảy ra sự cố:**

```
# Khôi phục trạng thái về đúng commit bị mất
git reset --hard c9d4e5f

# Xác thực kết quả khôi phục
git log --oneline
cat important.txt
```

- **Kết quả xác thực:**
  - *Commit 5: Add critical production feature* xuất hiện trở lại trên đỉnh của *git log*.
  - Tệp *important.txt* cùng nội dung "*CRITICAL PRODUCTION FEATURE - DO NOT LOSE*" được phục hồi nguyên vẹn 100%.

5. Tổng hợp so sánh các công cụ khôi phục:

Lệnh git	Phạm vi tác động	Có làm thay đổi lịch sử?	Trường hợp sử dụng tối ưu
git restore	Working Directory / Staging	Không	Hủy bỏ nhanh các dòng code mới gõ sai chưa kịp commit.
git stash	Working Directory / Staging	Không	Cất tạm code đang viết dở để chuyển sang nhánh khác xử lý gấp.
git revert	Repository (Commit)	Không (tạo commit mới)	Hoàn tác lỗi trên các nhánh dùng chung (main, develop) đã push lên Remote.
git reset	Repository / Staging / Working	Có (xóa bớt lịch sử)	Dọn dẹp các commit nháp trên nhánh cá nhân cục bộ trước khi push.
git reflog	Con trỏ HEAD cục bộ	Chỉ đọc (read-only)	Tra cứu mã hash để cứu các commit/nhánh bị mất do reset --hard hoặc xóa nhầm nhánh.

6. Kết luận và bài học rút ra:

- **Kết quả đạt được:** Thực hành thành công toàn bộ 5 công cụ khôi phục dữ liệu của Git (restore, stash, revert, reset, reflog), đặc biệt nắm vững quy trình cứu hộ commit bị mất sau sự cố git reset --hard.
- **Nguyên tắc an toàn cốt lõi:**
  - Tuyệt đối không dùng *git reset --hard* trên các nhánh đã chia sẻ cho nhóm; thay vào đó hãy dùng *git revert*.
  - Trong Git, một khi dữ liệu đã được commit, nó gần như không bao giờ bị mất ngay lập tức — *git reflog* chính là "phao cứu sinh" cuối cùng giúp khôi phục mọi sai lầm thao tác trên máy cục bộ.

HẾT!